

HYDRAS Project Report v9

1. Algoritmo PPO (Proximal Policy Optimization)

Proximal Policy Optimization (PPO) è un algoritmo di reinforcement learning **on-policy**, appartenente alla famiglia dei **Policy Gradient Methods**. La black-box è strutturata secondo uno schema **actor-critic**: una rete (policy) produce la distribuzione di probabilità delle azioni, mentre una seconda rete (value function) stima il valore dello stato $V(s)$.

Stable-Baselines3 (SB3) è una libreria open-source per reinforcement learning in Python, costruita su PyTorch. Fornisce implementazioni standardizzate e affidabili dei principali algoritmi (tra cui PPO), con API omogenee per training/inferenza, gestione dei callback, logging e ambienti vettorizzati. In pratica, SB3 è il framework che incapsula la black-box algoritmica e ne rende l'uso riproducibile e comparabile tra esperimenti.

In **Stable-Baselines3**, PPO è implementato con una policy neurale (tipicamente MLP) che elabora l'osservazione e produce due uscite: **policy head** e **value head**. In termini architetturali, il pattern generale è: input osservazionale, hidden layers fully-connected, output su spazio azioni e stima del valore.

Il meccanismo chiave è il **clipped surrogate objective**: PPO massimizza il miglioramento della policy, ma limita la deviazione rispetto alla policy precedente tramite clipping del rapporto $r_t(\theta) = \pi_\theta(a|s) / \pi_{\theta_{old}}(a|s)$. Questo vincolo riduce update troppo aggressivi e aumenta la stabilità dell'apprendimento.

L'obiettivo complessivo combina tre contributi:

- **Policy loss (clipped)**: aumenta la probabilità delle azioni vantaggiose;
- **Value loss**: allinea la stima $V(s)$ ai ritorni osservati;
- **Entropy bonus**: mantiene esplorazione, evitando policy premature/deterministiche.

Una forma compatta è: $L = L_{clip} + c1 \cdot L_{vf} - c2 \cdot H(\pi)$.

PPO usa tipicamente **GAE (Generalized Advantage Estimation)** per stimare il vantaggio con buon compromesso bias-varianza. In training la policy è stocastica (campionamento da $\pi(a|s)$); in inferenza può essere resa deterministica scegliendo l'azione a probabilità massima.

In questo progetto viene utilizzato **MaskablePPO** (da sb3-contrib), un'estensione di PPO che supporta **action masking**. Ad ogni step, le azioni che porterebbero l'agente su terra o fuori dal dominio vengono mascherate prima del campionamento dalla policy, impedendo fisicamente le collisioni con la costa e le uscite dal dominio. Questo rende il training più stabile, elimina penalità di terminazione evitabili e permette all'agente di focalizzarsi interamente sull'apprendimento della strategia di source seeking.

2. Dataset — 132 Sorgenti × 4 Scenari Vento

2.1 Acquisizione e Gestione Dati

Sorgente Dati: I dati di concentrazione provengono da simulazioni **MIKE21** in formato NetCDF (1411 timestep, risoluzione 10m, griglia 300×250 celle). Il modulo **data_loader.py** gestisce il caricamento automatico di **132 sorgenti** (SRC001-SRC132) × **4 scenari vento** (V0, V1, V2, V3): legge le coordinate spaziali (x, y), temporali (time) e i valori di concentrazione da file NetCDF, costruendo un campo interpolabile per ogni sorgente. Ogni sorgente ha coordinate geospaziali (UTM32N) caricate da **Coordinate_Sorgenti_Fasell.csv**.

Dataset Split: Dataset suddiviso in **80% training** (~106 sorgenti per scenario vento) e **20% valutazione** (~26 sorgenti held-out per scenario vento), equamente distribuiti tra i 4 scenari V0-V3.

Augmentazione Dati (Chunking): Per massimizzare la variabilità e creare multipli scenari di partenza per ogni sorgente, i 1411 timestep di ogni simulazione vengono suddivisi in **3 chunk temporali**:

- **Chunk 0** (Q1/4, spawn @ 352 timestep): inizio della propagazione del plume, concentrazione concentrata;
 - **Chunk 1** (Q1/2, spawn @ metà simulazione): stadio intermedio, plume parzialmente disperso;
 - **Chunk 2** (Q3/4, spawn @ 1058 timestep): stadio avanzato, plume pesantemente disperso da vento e correnti.
- L'agente può essere inizializzato in tre fasi diverse della dispersione, aumentando la robustezza del modello.

2.2 I Quattro Scenari Vento (V0, V1, V2, V3)

Il progetto utilizza **4 versioni diverse di vento** (CI_WIND_fasell_V0.txt, V1.txt, V2.txt, V3.txt), generati da simulazioni meteorologiche con parametri e risoluzioni differenti. Questo aumenta vastamente la diversità dei dati di training, permettendo al modello di generalizzare su condizioni vento realistiche. Ogni scenario ha il proprio file di vento (48 timestep) e le correnti oceaniche corrispondenti (estratti da CL02_V0/V1/V2/V3_SRC000_U_V_10mGrid.nc). Il training espone il modello a tutti e 4 gli scenari simultaneamente, creando una policy robusta a diverse condizioni meteorologiche.

2.3 Ottimizzazione: Caching In-Memory delle Correnti

Per evitare colli di bottiglia I/O durante il training, il modulo **data_loader.py** implementa un **sistema di caching in-memory** per i dati di vento e corrente:

Meccanismo di Cache: Al startup del DataManager, il metodo **_preload_all_versions_cache()** carica tutti e 4 i file di vento e corrente in RAM (dizionari Python):

- `_cached_wind_data = {"V0": array, "V1": array, "V2": array, "V3": array}`
- `_cached_current_data = {"V0": array, "V1": array, "V2": array, "V3": array}`

Durante il training, i metodi **get_wind_data_for_run()** e **get_current_data_for_run()** estraggono i dati dalla cache lookup-by-version, evitando riletture disk. Questo accelera ~5-6× il training rispetto a letture da disco: **~55K step/ora → ~300K step/ora**.

Sincronizzazione Garantita: La versione (V0/V1/V2/V3) viene estratta dal filename del file NC (es. CL02_V2_SRC042_Conc_10mGrid.nc → V2), quindi è **impossibile un mismatch** tra file di concentrazione, vento e corrente.

3. Ambiente di Simulazione

L'ambiente (**source_seeking_env.py**) è una griglia 300×250 celle (risoluzione 10m) che rappresenta un dominio marino di 3×2.5 km nella baia di Cecina (coste toscane, UTM32N). L'agente (AUV - Autonomous Underwater Vehicle) si muove con **8 azioni discrete**: N, S, E, W, NE, SE, NW, SW. La velocità è **1 m/s** e il timestep **dt=10s**, quindi ogni azione sposta l'agente di 10 metri (o ~7m per componente nelle direzioni diagonali). L'episodio termina quando la sorgente viene raggiunta (distanza < 50m) oppure dopo 1080 step (~3 ore simulate).

Evoluzione Temporale: Il campo di concentrazione evolve nel tempo: ogni 12 step dell'agente (~2 minuti simulati) il campo NetCDF avanza di 1 frame temporale. Vento e corrente sono sincronizzati in minuti simulati lungo tutto l'episodio.

Action Masking: Ad ogni step, MaskablePPO riceve una maschera booleana delle 8 azioni. Le azioni che porterebbero l'agente su terra o fuori dal dominio vengono invalidate prima del campionamento, impedendo terminazioni precoci per collisione. Se tutte le azioni sono mascherate (agente intrappolato), la maschera viene azzerata per evitare crash.

3.1 Constraints di Spawn e Fallback Automatico

Procedura di Spawn a Cascata: L'agente viene inizializzato su una cella del plume (concentrazione > 0.5) rispettando vincoli di distanza dalla sorgente e dalla costa. La procedura prova anelli concentrici in ordine decrescente di distanza, garantendo spawn sempre lontani dalla sorgente quando possibile:

Cascata di anelli (dal più lontano al più vicino):

- (2000–2500 m) → (1500–2000 m) → (1000–1500 m) → (500–1000 m) → (250–500 m) → (50–250 m) → (0–50 m)

Per ogni anello, vengono selezionate celle con **concentrazione > 0.5** e distanza dalla terra ≥ 50 m. La cascata scende all'anello successivo solo se nessun punto del plume rientra in quello corrente. **Fallback finale:** se nessun anello ha celle valide, lo spawn avviene ovunque nel plume senza vincoli di distanza.

4. Training e Architettura

4.1 Input della Rete Neurale — 116 Dimensioni

Lo spazio di osservazione è un vettore continuo a **116 dimensioni** (normalizzato via VecNormalize). Include la memoria storica delle concentrazioni, sensori radiali nelle 8 direzioni e — da questa versione — memoria chimica estesa con posizione del massimo e conteggio del contatto col plume.

Parallelizzazione: il training usa **SubprocVecEnv** con **6 ambienti paralleli** (2 worker × 3 chunk), avviati tramite start_method='fork' per ereditare il DataManager già caricato in RAM senza ricostruirlo in ogni sottoprocesso.

Componente	Dim	Descrizione
Concentrazione Attuale	1	Conc. posizione corrente (x, y)
Memoria Conc. (Locale)	9	Ultimi 9 timestep
Storico Movimento	18	$\Delta x, \Delta y$ ultimi 9 step
Sensori Radiali (Correnti)	8	Conc. nelle 8 direzioni a distanza 20m
Memory Conc. Direzionali	72	Conc. nelle 8 direzioni × 9 timestep passati
Vento	2	u, v [m/s]
Corrente	2	u, v [m/s]
Max Conc. Vista	3	(dx, dy) verso la posizione di massima concentrazione osservata nell'episodio + valore massimo normalizzato
Steps fuori Plume	1	steps_since_plume_contact / max_steps — misura quanto a lungo l'agente è fuori dal plume

4.2 Parametri di Training

Parametro	Valore
Algoritmo	MaskablePPO (sb3-contrib) + action masking
Architettura Policy	MlpPolicy — MLP [256, 256], attivazione Tanh (actor e critic)
Normalization	VecNormalize (osservazioni, clip_obs=10)
Batch Size	128
N Steps	4096
N Epochs	10
Learning Rate	3e-4 (0-2M) -> 1e-4 (2-4M) -> 5e-5 (4-5M) -> 1e-5 (5-6M)
Clip Range	0.15
Target KL	0.01
Gamma	0.99
GAE Lambda	0.95
Entropy Coef	0.01
VF Coef	0.3
Timestep Totali	6.0M (6,000,000)
Curriculum Scenari	Scenario curriculum 3 fasi: 0–2M: uniforme → 2–4M: 50/50 hard/easy → 4–6M: 80/20 hard/easy
Ambienti Paralleli	2 workers × 3 chunk = 6 env simultanei (SubprocVecEnv)

4.3 Funzione di Reward

Componente	Valore/Descrizione
------------	--------------------

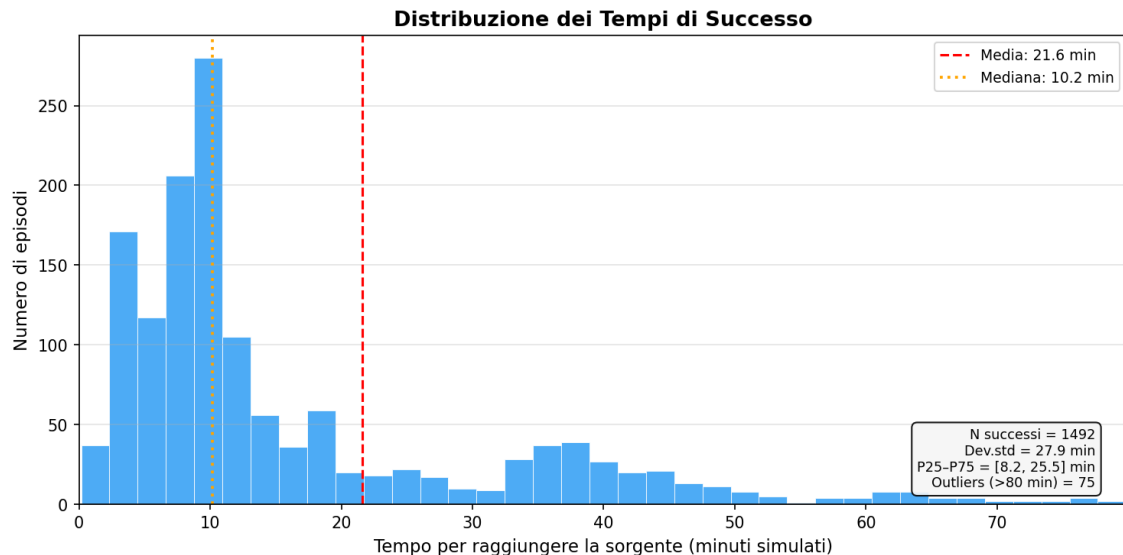
Sorgente Raggiunta	+100.0 + time_bonus (max +50)
Penalità Bordo	-10.0
Reward Distanza	delta_phi x zone_mult x 100 — potenziale quadratico $\phi(d)=(3000-d)^2/3000^2$
Zone Multiplier	max(1, 2000/d) — continuo: 1x a d>2000m 2x a 1000m 20x a 100m 40x a 50m
Retreat Asymmetry	x3 se allontanamento — scoraggia bypass della sorgente
Gradiente Conc.	+0.05 se conc. aumenta / -0.05 se diminuisce
Vento	+0.05 controvento (upwind) / -0.05 a favore (downwind)
Corrente	+0.05 controcorrente (upstream) / -0.05 a favore (downstream)
Penalità Tempo	-0.1/step
Vicinanza Terra	max -15.0 (progressiva entro 30 m dalla costa)
Stagnazione	-1.5 se delta_dist < 20m in 15 step — sospesa entro 250m dalla sorgente
Stagnazione Direzionale	-0.5 se efficienza < 0.15 (oscillazione) — sospesa entro 250m

5. Analisi Quantitativa dei Risultati

Analisi dettagliata condotta su **1560 episodi** raccolti durante l'inferenza del modello **ppo_20260513** (evaluations_v12) (1492 successi, 68 fallimenti). I dati provengono da: *evaluations_v12/episodes_data.json*.

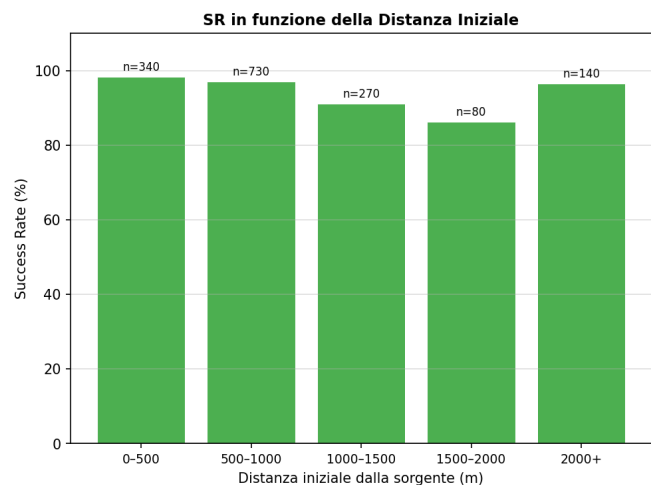
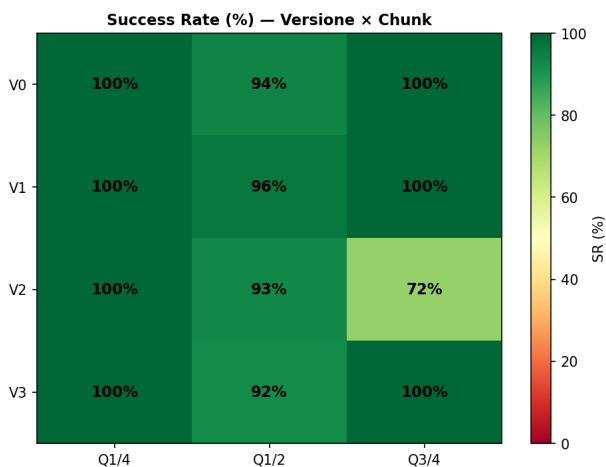
5.1 Distribuzione dei Tempi di Successo

Istogramma dei tempi simulati impiegati dall'agente per raggiungere la sorgente, limitato al 95° percentile dei successi per evidenziare la struttura principale della distribuzione. Il box in basso a destra riporta le statistiche complete inclusi gli outlier. La distribuzione è asimmetrica a destra: la maggior parte dei successi avviene entro 20 minuti simulati, con una coda dovuta a scenari più complessi (plume disperso, spawn lontano).



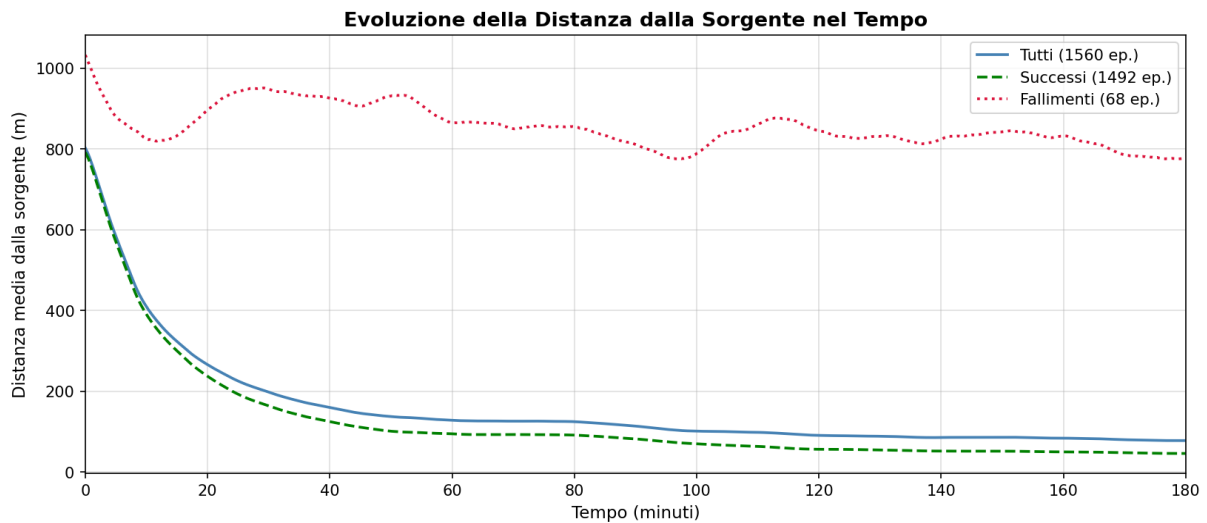
5.2 Successo Atteso in Funzione dello Scenario

Sinistra: heatmap della success rate per ogni combinazione versione vento × chunk temporale. Evidenzia quali scenari sono strutturalmente più difficili (V2, chunk avanzati). **Destra:** success rate in funzione della distanza iniziale di spawn dalla sorgente (bins da 500 m).



5.3 Distanza dalla Sorgente nel Tempo

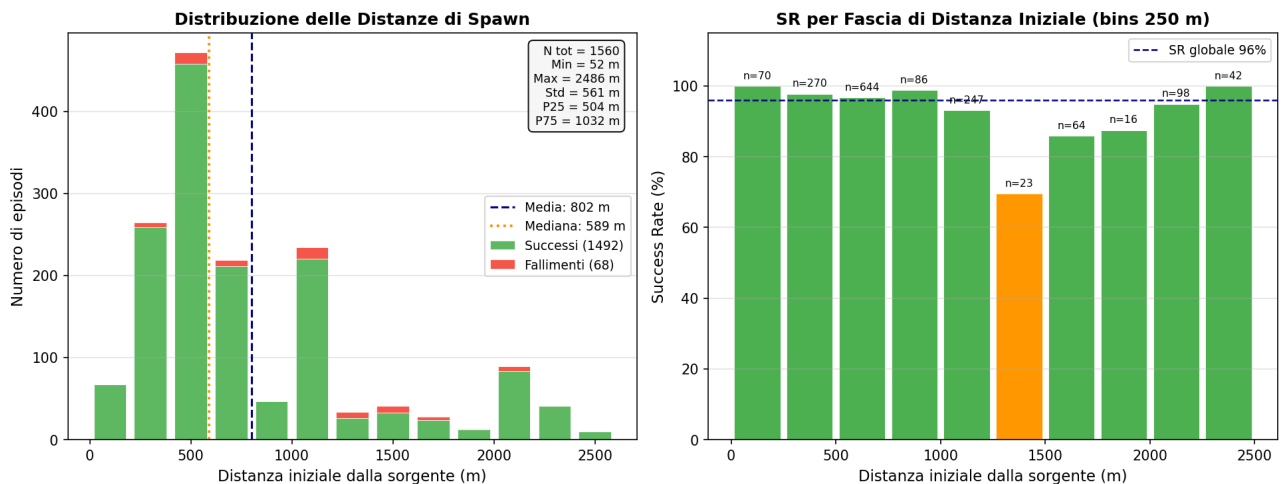
Distanza media dalla sorgente in funzione del tempo simulato (in minuti), mediata su tutti gli episodi. Le curve separate per successi e fallimenti mostrano il diverso comportamento di convergenza: gli episodi di successo mostrano una riduzione progressiva e sostenuta della distanza, mentre i fallimenti tendono a oscillare o stabilizzarsi a distanze elevate.



5.4 Distribuzione della Distanza Iniziale di Spawn

Sinistra: istogramma delle distanze iniziali di spawn (bins da 200 m) con sovrapposizione di successi (verde) e fallimenti (rosso). La procedura di spawn a cascata garantisce una distribuzione ampia e variabile delle distanze di partenza, evitando che il modello sia valutato sempre alla stessa distanza dalla sorgente. La coda destra mostra che il modello gestisce efficacemente anche spawn molto lontani (oltre 2000 m).

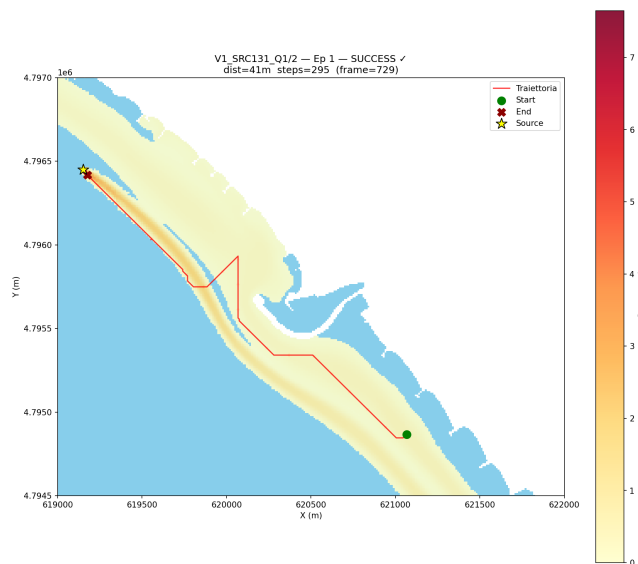
Destra: success rate per fascia di distanza iniziale (bins da 250 m). Permette di identificare se la difficoltà del task scala con la distanza di spawn: la SR rimane alta e stabile su tutto il range, confermando che la policy generalizza indipendentemente dal punto di partenza.



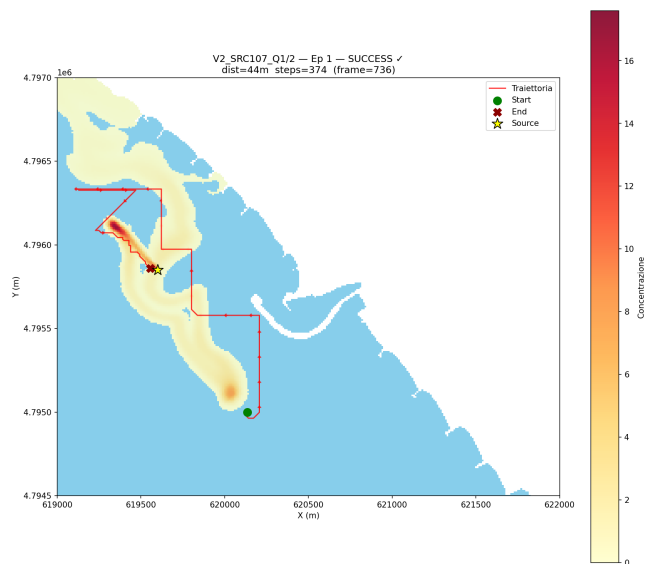
6. Analisi Visiva dei Risultati

Confronto tra traiettorie di episodi rappresentativi dell'inferenza condotta sul modello **ppo_20260513** (evaluations_v12). Le immagini mostrano il percorso dell'agente (linea colorata) sovrapposto al campo di concentrazione del plume chimico e alla posizione della sorgente (stella rossa).

6.1 Casi di Successo

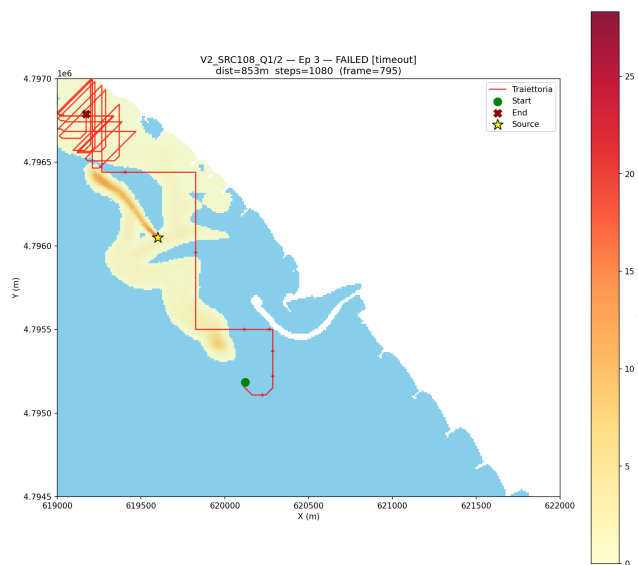


Successo — SRC131, Vento V1 (Q1/2), dist. iniziale 2486 m, 295 passi (~49 min sim.)

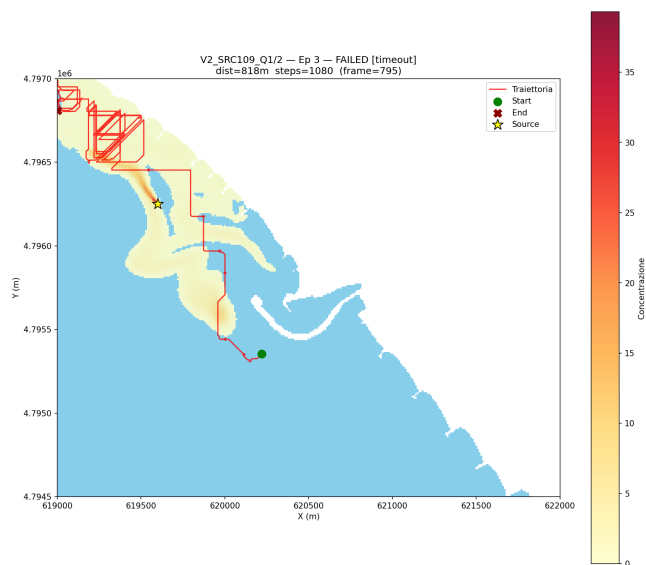


Successo — SRC107, Vento V2 (Q1/2), dist. iniziale 1006 m, 374 passi (~62 min sim.)

6.2 Casi di Fallimento



Fallimento — SRC108, Vento V2, dist. iniziale 1011 m, timeout (1080 passi)



Fallimento — SRC109, Vento V2, dist. iniziale 1091 m, timeout (1080 passi)